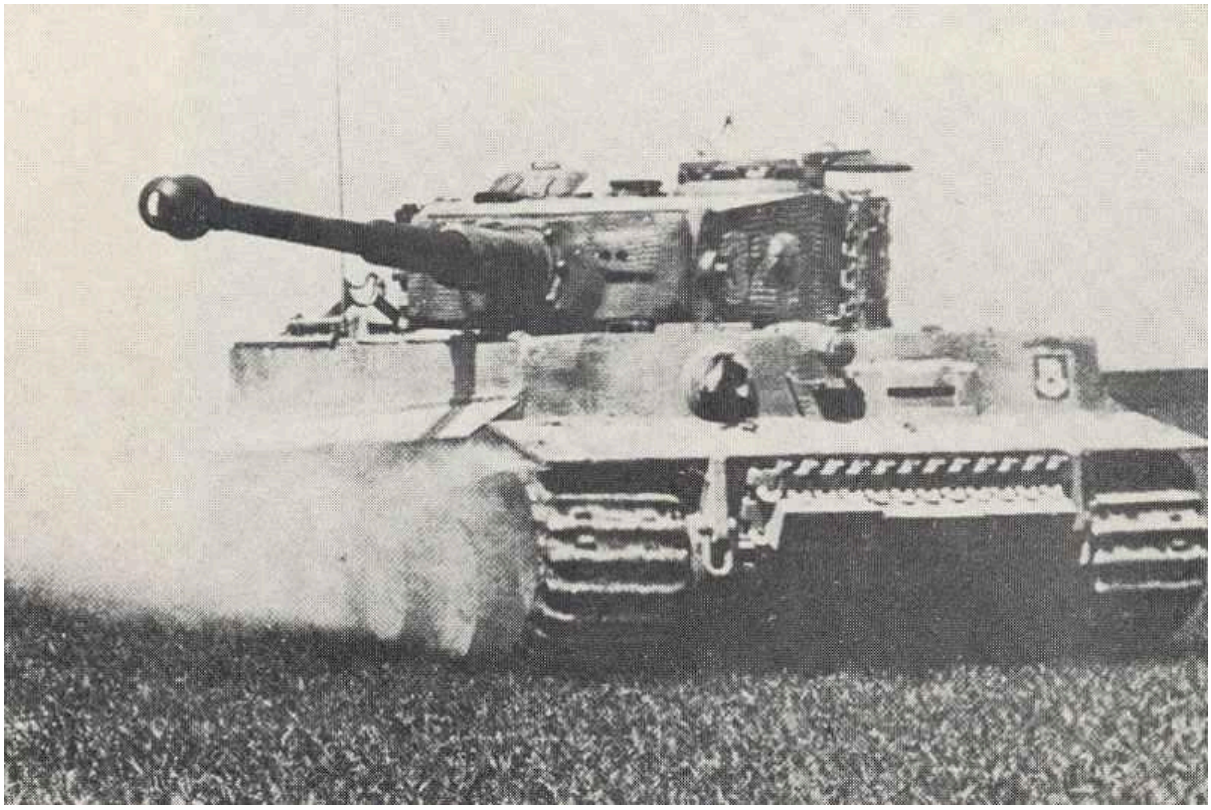
Projektbericht “Panzerfahrersimulator”

Gruppe 05
Praktikum Virtual Reality (B084)
René Graap, cgt105587
Nico Müller, cgt105427



Tiger I der Pz.Abt.502 im Kampfeinsatz im Nordabschnitt der Ostfront (Frühsommer 1944).
https://www.weltkrieg2.de/tiger-bataillone/

Einleitung:

Aufgrund eines gehaltenen Seminars “VR in der Rüstungsindustrie” und der dabei erwähnten Verwendung von VR als Trainingssimulation, kamen wir auf die Idee, einen Simulator für das Panzerfahren zu entwickeln.
Anfangs war der Plan, einen oberflächlichen Simulator zu entwickeln, der die Ausübung von vielen verschiedenen Tätigkeiten ermöglicht. Der Simulator war aber dementsprechend auch nie dafür vorgesehen, potenziell tatsächliches Panzerfahren daran zu trainieren. Vielmehr sollte dem Nutzer ein vereinfachter, aber immer noch authentischer Eindruck der Tätigkeiten in einem Panzer vermittelt werden.
Da das Ganze auch einen gewissen spielerischen Anreiz bieten sollte, bei dem die Möglichkeiten von VR auch zum Einsatz kommen, haben wir uns früh dazu entschieden, dass der Simulierte Panzer ein älteres Panzermodell sein sollte. In modernen Panzern funktioniert viel automatisch und der Nutzer muss nur noch einen Knopf drücken. Der Nutzer sollte die Möglichkeiten bekommen auch an Hebeln zu ziehen und Kurbeln zu drehen.
Nach einer Anfrage beim deutschen Panzermuseum Munster, wurde uns der Panzerkampfwagen VI bzw. der “Tiger” empfohlen, da es dazu am meisten Literatur zu finden gibt. In den auszuübenden Tätigkeiten und Funktion unterscheiden sich die Panzer vor dem Kalten Krieg nicht sonderlich stark. Daher folgten wir dem Rat des Panzermuseums und wählten den Tiger als Simulationsmodell aus.
Bei der Recherche stießen wir ziemlich schnell auf ein Problem - den Umfang des Simulators. Wenn man die Tätigkeiten der Panzerbesatzung detailliert abbilden wollte, würde der Aufwand des Projekts den geplanten Arbeitsumfang im Rahmen des Praktikums deutlich überschreiten. Daher standen wir vor der Wahl, die einzelnen Tätigkeiten der Besatzung stark zu abstrahieren oder den quantitativen Umfang des Simulators einzuschränken, also nur noch ein Besatzungsmitglied detailliert zu simulieren. Wir entschieden uns für die zweite Option, da wir lieber Qualität statt Quantität produzieren wollten.
Da die erste Tätigkeit, die bei einem Kraftfahrzeug sinnvoll simuliert werden sollte, das Fahren ist, werden in dem Simulator die Panzerfahrer Tätigkeiten simuliert.

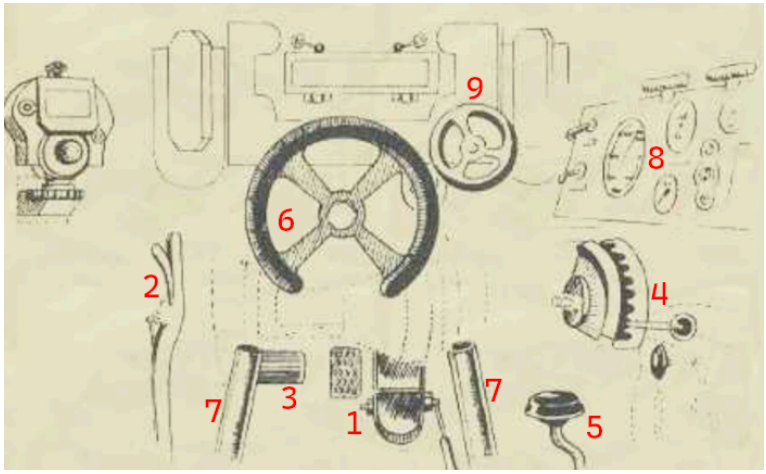
In dem Programm bekommt der Nutzer kein künstliches Ziel, welches ihn zum Spielen motivieren soll. Vielmehr soll es eine lehrreiche Erfahrung sein, bei der der Nutzer erfährt, in welcher Umgebung ein Panzerfahrer im Einsatz ist. Dabei spielen aus unserer Sicht vor allem drei Aspekte eine wichtige Rolle - das Sichtfeld des Panzerfahrers, die Lautstärke und die Koordination beim Bedienen mehrerer Hebel oder Ähnlichem.

Projektorganisation:

Geplante Mechaniken:

Nachdem wir die simulierten Tätigkeiten auf die des Fahrers eingeschränkt hatten, wurden folgende Tätigkeiten zur Simulation geplant:

1. Das Gasgeben und Bremsen per Pedale
   Beschleunigen und Abbremsen des Fahrzeugs. Bei angezogenem Kettensteuerhebel (7) wirkt die entsprechende Beschleunigung nur auf eine Kette.
2. Das Bremsen mit Handbremse (Hebel)
   Blockiert beide Ketten.
3. Das Kuppeln mit Pedal
   Entkuppelt beim Betätigen, sodass der Gang (4) und/oder Richtungshebel (5) verändert werden können.
4. Das Einstellen des Ganges per Schalthebel
   Ermöglicht das Umschalten der Gänge. Der Tiger Panzer besitzt 8 Vorwärts- und 4 Rückwärtsgänge.
5. Das Einstellen der Fahrtrichtung per Richtungshebel
   Legt die Fahrtrichtung fest (Vorwärts, Leerlauf, Rückwärts).
6. Das Lenken während der Fahrt mit Lenkrad
   Lenkt das Fahrzeug, während beide Ketten betrieben werden.
7. Das Lenken im Stand (zum Drehen auf der Stelle) mit zwei Kettensteuerhebeln
   Bei Betätigung wird die entsprechende Kette blockiert, sodass das Fahrzeug sich (fast) auf der Stelle dreht. Sollte nur im Stand betätigt werden.
8. Ablesen von Technischen Daten aus der Anzeige
   Zeigt wichtige technische Daten an. Für das direkte Fahren sind jedoch nur Geschwindigkeit und Drehzahl wirklich relevant.
9. (Optional: Öffnen und Schließen der Fahrerluke und des Sichtschlitzes)
   Durch das Drehen an der Kurbel lässt sich der Sehschlitz öffnen und schließen. Durch Greifen und Drücken/Ziehen gegen die Fahrerluke, lässt sich diese öffnen und schließen.
   Leider haben wir diese Mechaniken aus zeitlichen Gründen nicht mehr geschafft



Abgeändertes Bild aus “Die Tigerfibel - Handbuch des deutschen Tiger Panzers” Seite 40,
https://www.lexikon-der-wehrmacht.de/Waffen/Merkblatt/Die_Tigerfiebel_Handbuch_des_deutschen_Tiger_Panzers_.pdf

Bedienungselemente:

Fast alle Tätigkeiten sind motorische Mechaniken, die sich einfach in VR umsetzen lassen. Die einzige Ausnahme bildet hierbei das Betätigen der Pedalen. Die meisten VR Setups bieten keine Tracking-Controller für die Füße an. Daher war hierfür der Input per Pedal (Logitech G29) geplant. Leider müssen die Pedalen über das dazugehörige Lenkrad angeschlossen werden, obwohl dieses für unser Programm keinen Nutzen bietet. An dieser Stelle gab es zwar die Überlegung, ob man das Logitech Lenkrad für den Lenk-Input der Panzers einbindet, allerdings müssten der Nutzer dann, während der Anwendung zwischen Lenkrad und VR-Controllern wechseln, welches von uns als zu unpraktisch angesehen wurde. Daher wurde diese Idee wieder verworfen und sich auf die Nutzung der Pedalen beschränkt.

Alle Bedienelemente, welche mit den Controllern bedient werden, mussten nur das Greifen und das Bedienen des Menüs implementieren. Dies wurde schon großteils von der Engine übernommen, sodass von unserer Seite immer nur noch die Reaktion auf einen Griff implementiert werden musste.

Die optionalen Bedienelemente für das Öffnen und Schließen der Sicht- oder Fahrerluke konnten aus Zeitgründen leider nicht mehr implementiert werden.

Menü:

Es war geplant, das Menü auf die Wand links vom Fahrer zu projizieren, so dass der Spieler diese gut erreichen kann, aber nicht andauernd das Menü im Blickfeld hat. Alternativ gab es optional die etwas immersivere Idee, das Menü in einem Handbuch anzuzeigen, welches in einer Halterung an der Wand hängt. Diese Idee fanden wir besser und haben diese auch versucht umzusetzen. Anfangs wollten wir es komplett als physisch korrekt simuliertes Objekt im Panzer haben, allerdings stießen wir dabei auf mehrere Probleme. Erstens führte es zu einigen Problemen mit der Unreal Engine, da die Kollision nicht korrekt erkannt wurde. Zweitens und noch viel wichtiger war aber die Tatsache, dass der Nutzer das Handbuch “verlieren” konnte. Wenn das Handbuch aus der Reichweite des Nutzers rutschte oder die Orientierung des Nutzers falsch eingestellt war, kam der Nutzer nicht mehr ans Handbruch ran und konnte somit nicht mehr das Menü aufrufen oder bedienen. Daher wurde es so abgeändert, dass wir ein robustes Menü in der Hand erzeugen und nicht die Instanz von der Wand nehmen.
Da wir die immersive Idee des greifbaren Menüs nicht verwerfen wollten, aber unter anderem das Problem der Neuorientierung hatten, haben wir den Kompromiss gemacht, dass sich das Menü über zwei Möglichkeiten aufrufen lässt. Entweder kann der Nutzer das Menü aus der Halterung an der Wand “nehmen” (immersiver Ansatz) oder aber er kann es an seinem Controller über die Daumentaste öffnen. Damit kann der Nutzer in jeder Situation das Menü erreichen und bedienen.
Ansonsten hat die Umsetzung des bedienbaren Handbuchs sehr gut funktioniert und passt zur Umgebung des Programms.

Level:

Das Leveldesign war von Anfang an eine optionale Anforderung, weshalb es keine hohe Priorität in der Umsetzung hatte und erst spät ins Programm eingebunden wurde. Allerdings gab es dennoch früh einige Ideen, was wir für ein Level konzipieren wollen, beziehungsweise was für Elemente sich gut eignen, um den Simulator in seiner Funktionalität zu unterstützen.
Die grundsätzliche Thematik des Levels sollte ein Übungsgelände sein. Dabei sollte das Leveldesign es sinnvoll ermöglichen, alle implementierten Mechaniken des Panzers zu nutzen. Daher wurde eine lange offene Fläche benötigt, auf der man den Panzer auf seine Höchstgeschwindigkeiten beschleunigen kann und dann auch noch fahren kann. Zudem sollte es aber auch enge Bereiche geben, in denen der Nutzer langsam fahren muss und mit der starken Lenkung und dem Rückwärtsgang arbeiten muss. Dafür bot sich etwas wie ein “Grabenlabyrinth” oder ein eng bebautes Siedlungsgebiet an.
Ebenso war es wichtig, dass es auch Stellen gibt, die das Verhalten des Panzers im Gelände darstellen. So sollte es Stellen geben, an denen man den Schwerpunkt des Panzers erfahren soll. Dafür eignet sich ein Hindernis wie eine Rampe, über die man fahren kann, oder ein hügeliges Gelände-Areal.
Auch bieten sich Designelemente an, welche die Funktionalität eines Panzers im Krieg erklären. Ein wichtiger Einsatzaspekt von großen Panzern ist es, dass sie Schützengräben überfahren können und so im Kampfgebiet mobil bleiben. Auch solche Dinge lassen sich gut zeigen und darstellen.

Panzergestaltung:

Da das Programm als Simulator geplant ist, muss auch die Ausgestaltung des Panzers in den essentiellen Aspekten historisch akkurat sein. Daher wurden Baupläne des Panzerkampfwagen VI als Referenz benutzt, um ein glaubwürdiges Design zu erstellen.
Leider mussten viele Assets von uns selbst modelliert werden, da uns keine 3D-Scans von einem Tiger Panzer zur Verfügung standen.
Besonders beim Design des Innenraums mussten wir improvisieren, da es wenig/keine Bilder vom Innenraum eines Tiger Panzers gibt. Das Design wurde daher vom Innenraum anderer Panzermodelle abgeleitet. Eine Ausnahme dabei bildet das Design der Steuerelemente. Für das Design dieser Elemente wurde eine Skizze aus einem Tiger Panzer-Lehrbuch (siehe Geplante Mechaniken) herangezogen.
Ansonsten wurde versucht möglichst viele Modelle und Texturen aus dem Fab Shop oder von Quixel Bridge zu integrieren, um den Arbeitsaufwand zu verringern.

Fahrzeugsteuerung:

Generell:
Für eine realitätsnahe Funktionsweise des Fahrzeugs müssen mehrere Kfz-Untypische Mechaniken implementiert werden. Als Basis konnte zwar die ChaosWheeledVehicle (CWV) API von der Unreal Engine genutzt werden, allerdings gibt es mehrere Probleme mit dieser Schnittstelle. Der Vorteil durch die Nutzung der API ist, dass wir einen großen Teil der Physik Berechnung des Fahrzeugs der Engine überlassen konnten. Allerdings ist Fahrzeugschnittstelle für Rennspiele und andere oberflächliche Fahrzeugbedienung ausgelegt und sieht nicht vor, mehrere Arten von Handbremsen zu implementieren oder eine Lenkung ohne “lenkende” Reifen zu besitzen. Diese Abnormalitäten zu implementieren hat mitunter am meisten Zeit benötigt - besonders, da diese im Zusammenspiel mit der vorgegebenen API sehr Bug anfällig waren.

Gangschaltung:
Eine weitere Besonderheit beim Panzerkampfwagen VI im Vergleich zu einem Auto ist die aufgeteilte Gangschaltung. In einem Auto mit manueller Gangschaltung existiert nur ein Schalthebel, der direkt für den Fahrer definiert, wie viele Vorwärtsgänge und Rückwärtsgänge das Fahrzeug besitzt. Der Panzer hingegen besitzt eine höhere Anzahl an Gängen - insgesamt 12 Gänge und eine Leerlaufstellung.
Die CWV API sieht außerdem vor, die Gangschaltung automatisch und nicht manuell zu verändern. Da wir aber eine manuelle Gangschaltung mit zwei Hebeln benötigen, entsteht eine Reihe an Konstellationen, die wir entweder unterbinden oder so handhaben müssen, dass der Nutzer diese möglichst nicht verwendet. So kann der Nutzer beispielsweise in einen Rückwärtsgang wechseln, obwohl er im 8. Vorwärtsgang ist. Da der Panzer aber nur 4 Rückwärtsgänge besitzt, würde dann eine unzulässige Konstellation erzeugt werden. In unserer Literatur wurde nicht eindeutig definiert, was bei einem solchen Fehlverhalten passiert. Dort hieß es nur, dass vor dem Schalten, in den Rückwärtsgang, in den 4. Gang runtergeschaltet werden soll.
Je nach Einzelfall mussten wir entscheiden, was das plausibelste Verhalten des Fahrzeugs wäre. So haben wir uns in dem eben erwähnten Fallbeispiel dazu entschieden, dass das Getriebe direkt kaputt geht, wenn man in einem zu hohen Gang in den Rückwärtsgang schaltet, da diese Konstellation nicht möglich sein darf. Wenn der Fahrer allerdings korrekt in den Rückwärtsgang geschaltet hat, kann der Hebel nur noch bis maximal in den 4. Gang geschaltet werden. Dieser Zustand gilt, bis er den Richtungshebel wieder auf Leerlauf oder Vorwärtsgang verändert.
Ein weiterer Fall war das Verhalten beim Schalten, ohne das Kupplungspedal zu drücken und dadurch das Fahrzeug zu entkuppeln. Theoretisch war es möglich, ohne das Kupplungspedal zu betätigen, den Panzer zu fahren. Dieses Fehlverhalten führt bei Fahrzeugen nicht direkt zu einem Totalschaden, jedoch zu starkem Verschleiß der Kupplung. Daher haben wir für solche Verschleiß-Fälle eine weitere Mechanik implementiert. Die Mechanik ist ein “Fehler”-Counter. Damit darf der Nutzer mindestens ***XX*** Fehler machen. Sobald mehr Fehler gemacht werden, wird bei jedem Fehler eine Wahrscheinlichkeit geprüft, ob der Schaden zu stark ist und zu einem Ausfall des Panzers führt. Die Wahrscheinlichkeit steigt proportional mit der Anzahl der gemachten Fehler. Bei 100 gemachten Fehlern fällt das Fahrzeug garantiert aus.

Durch diese Mechanik wird der Nutzer zwar für Fehlverhalten bestraft, ohne aber direkt neu starten zu müssen. Besonders, wenn der Simulator von Personen benutzt wird, die keine Erfahrung mit dem Fahren von Autos oder Ähnlichem haben (siehe Testing), würde hier eine zu starke Bestrafung zu Frustration führen. So haben wir eine Lösung gefunden, die den Nutzer zwingt, das Fahrzeug korrekt zu bedienen, ohne den unerfahrenen Nutzer zu hart zu bestrafen und einigermaßen realistisch ist, ohne das Getriebe bis ins Detail zu simulieren.

Lenkung:
Die Lenkung per Lenkrad war ohne große Probleme zu implementieren. Das Einzige, was beachtet werden musste, ist, dass ein Panzer keine Lenkachse besitzt. Die Lenkung entsteht über die Geschwindigkeitsdifferenz der beiden Ketten zueinander. Daher konnten wir nicht den Standardinput für die Lenkung verwenden, sondern mussten die Lenkung über die Rotation des Modells vollführen. Dies führt zu korrektem Verhalten bei der Rotation auf der Stelle, allerdings zu einem leichten Fehlverhalten bei Rotation in der Fahrt. Eigentlich wäre hierbei die Rotationsgeschwindigkeit abhängig von der gesamten Fahrzeuggeschwindigkeit, jetzt ist die Rotation aber losgelöst von dieser. Dies ist leider ein Fehler, allerdings die beste uns bekannte Lösung für die Lenkung, ohne die CWV API komplett zu verändern.
Die Rotation auf der Stelle kann zusätzlich auch zu Fehlverhalten führen. Die Rotation wird über das Anziehen eines Kettensteuerhebels erreicht. Diese blockieren die jeweilige Kette. Demnach funktionieren die beiden Kettensteuerhebel in Kombination als Handbremse. Die Problematik tritt auf, wenn nur ein Hebel während der Fahrt angezogen wird. Es ist vorgesehen, dass die Kettensteuerhebel nur während des Stehens angezogen werden. Wenn sie hingegen in der Fahrt angezogen werden, wird die Bremse komplett aus ihrer Verankerung gerissen und zerstört. Eine Kette wiegt mehr als 3 Tonnen und bewegt sich mit der doppelten Geschwindigkeit des Panzers. Die Kräfte, die auf die Bremse wirken, sind also enorm. Daher haben wir uns in so einem Fall auch für einen sofortigen Totalschaden entschieden. Dies ist zwar nicht realistisch, da eigentlich nur die Bremse ausfallen würde, aber da lag die Priorität erst einmal woanders, als weitere Schadensarten zu implementieren.

Anzeige von Daten:
Von Anfang an war geplant, auf dem Display die zwei wichtigsten technischen Daten per Rotationsnadel anzuzeigen - zum einen die Geschwindigkeit und zum anderen die Drehzahl. Beides ließ sich einfach umsetzen und funktioniert wie geplant. Die weiteren Anzeigen, Öl und Treibstoff, dienen nur der Dekoration. Es ergibt keinen Sinn hier Mechaniken einzubauen, die diese Ressourcen verbrauchen, ohne dass irgendein Nutzer den Simulator so lange am Stück testen würde, als dass sie leer gehen könnten. Es gab Überlegungen, dass der Treibstoff mit der Zeit verbraucht werden könnte und man dann wieder tanken muss. Allerdings bot es für die Anwendung keinen relevanten Mehrwert und wurde daher nicht integriert. An diesem Punkt könnte aber angeknüpft werden, sollten wir den Simulator weiterentwickeln.
Eine Weitere Anzeige die wir eingebaut haben, obwohl sie nicht in der Form im realen Panzer vorkam, ist die Schadenslampe. Es gab natürlich Kontrolllampen auch im Tiger Panzer, allerdings nie für einen gesamten Schaden. An dieser Stelle haben wir aufgrund von Benutzerfreundlichkeit und Arbeitsaufwand uns dafür entschieden, den Schaden erst einmal universell zu behandeln und anzuzeigen.

Ein weiterer Punkt, der von Anfang an geplant war, ist die Beschriftung des Kupplungshebel. Der aktuelle Gang ist eine relevante Information und war daher immer geplant, obwohl der beschriftete Schaltkasten erst gegen Ende eingefügt wurde. Dies lag mitunter daran, dass die Information für Testzwecke nicht nötig war und der Aufwand für die Einbindung sehr gering ausfiel. Dennoch wurde es verständlicherweise bei den Playtests angemerkt (siehe Testing).

Arbeitsteilung:

Fast alle Aspekte des Projekts wurden gemeinsam bearbeitet. Da wir eine kleine Gruppe waren, war das möglich. Zeitweise haben wir dadurch sicherlich etwas Arbeitspotenzial verschenkt, da zwei Leute sich an einem Problem die Zähne ausgebissen haben, allerdings hatte es den Vorteil, dass wir schneller auf Lösungen kamen und wir beide alle Teile des Programms gut kennen. Dies vereinfachte eine flexible Arbeitsweise, bei der jeder Probleme in unterschiedlichen Bereichen schnell lösen konnte. Außerdem konnten beide so in vielen Bereichen der Unreal Engine Erfahrung sammeln.
Die komplette Logik des Simulators wurde gemeinsam gebaut. Nicht jede einzelne Funktion wurde gemeinsam geschrieben, aber doch in enger Zusammenarbeit und Absprache.
In den letzten Wochen haben wir vermehrt getrennt gearbeitet, da wir vor allem am “Look and Feel” gearbeitet haben. So kümmerte sich René beispielsweise um die Ausgestaltung des Panzer Innenraums, während Nico sich um das Mapdesign kümmerte. Wenn wir einmal nicht zusammen an einem Problem gearbeitet haben, saßen wir dennoch gemeinsam im VR Labor und haben uns gegenseitig zugearbeitet.
Die enge Zusammenarbeit hat dafür gesorgt, dass wir unsere Repository Verwaltung zentral machen konnten und nicht mehrere Branches verwalten und zusammenführen mussten.

Die Arbeitspakete lassen sich folgend Kategorisieren:

- Konzeptionierung/Planung
- Modelling/Design - Panzer
- Leveldesign
- Logik Implementation
- Bugfixing
- Sonstiges (Berichtschrieben, Playtestauswertung, ect.)

Wie schon oben genannt, haben wir alle Aufgabenbereiche gemeinsam behandelt und gegebenenfalls nur Schwerpunkte verteilt.
Zu Beginn der Entwicklung ging es nur darum, erst einmal ein grobes Design von den wichtigsten Elementen wie Hebeln, Lenkrad und Panzerchassis vorzunehmen. Als nächstes wurde sich darum gekümmert, dass alle Inputs richtig im Programm ankommen und die richtigen Reaktionen auslösen. Die meiste Zeit hat die Logik Implementation gekostet. Besonders das Bugfixing musste iterativ immer wieder im Projektverlauf stattfinden. Am Ende, nachdem die Logik gut funktionierte, wurde sich vor allem dem Ausmodellieren von Panzerelementen und dem Mapdesign gewidmet.
Da die zwischenzeitig gemachten Playtests keine relevanten Änderungen an unserem Plan forderten, konnten wir diesen größtenteils auch gut umsetzen.

Testing:

Unser Testing bestand aus mehreren Durchläufen. Es gab zwei große Testphasen mit externen Leuten (nicht aus dem Entwicklerteam). Daneben haben wir natürlich alle unsere Mechaniken wiederholt selbst getestet. Insgesamt wurde durch die größeren Testphasen nur unsere eigene Testerfahrung bestätigt, weshalb wir keine großen Änderungen vornehmen mussten. Wir konnten uns also auf unsere Agenda konzentrieren, die wir schon von vornherein aufgestellt hatten.

Erster Testdurchlauf:

Im ersten Testdurchlauf waren Matteo di Sciullo, Steffen Fietkau, Curtis Radel, Luisa Teichert und Jule Corves unsere Tester.

Stand des Builds:

- die meisten Fahrzeuge Mechaniken (Lenken, Beschleunigen, Bremsen, Schalten per Hebel) waren implementiert
- Kupplung(-spedal) hat noch keine Relevanz
- Technische Daten wurden nicht angezeigt
- Innenraum des Panzers war nicht weiter ausgestaltet (Aktueller Gang nicht erkennbar)
- Menü war das Standardmenü von der Unreal Engine
- sehr minimalistisches Testlevel (nur Rampe und ein paar Unebenheiten)
- Erklärung der Steuerung in der README
- Fahrzeug kann nicht kaputt gehen

Die Ergebnisse des ersten Testlaufs waren sehr durchmischt. Zwar gab es positives Feedback für Dinge wie, dass der Panzer vernünftig auf die Inputs reagiert (z.B. sich die Pedalen bewegen, wenn das Pedal in Echt gedrückt wird) oder das sich die Fahrzeugphysik gut anfühlt, allerdings gab es auch viel Kritik.

Neben einigen neu entdeckten Bugs, welche wir im Anschluss beheben konnten, wurde vor allem die undurchsichtige Steuerung bemängelt. Viele Tester kamen mit den vielen Hebeln nicht zurecht, sodass einige sogar gar nicht erst testen konnten. Daher haben wir als nächstes dafür gesorgt, dass die Steuerung ingame erklärt wird. Dies haben wir mit dem Menü zusammen in ein Handbuch geschrieben, welches der Fahrer sich ingame durchlesen kann.

Außerdem hatten mehrere Tester das Problem, dass sie durch den Sichtschlitz kaum etwas sahen, weshalb wir auch die Kameraposition im Panzer etwas verändert haben. Aufgrund der schlechten Sichtbarkeit haben viele Leute Gebrauch von dem implementierten Debug-Button gemacht. Dieser war eigentlich nur für die Entwicklung vorgesehen, wurde jedoch von vielen Nutzern während der Fahrt verwendet. Daher haben wir auch zusätzlich eine geöffnete Luke eingebaut, sodass der Fahrer während der Fahrt mehr sehen kann, wenn er aus der Luker herausschaut.

Die meisten Probleme mit dem Input und dem Tracking konnten wir auf externe Faktoren wie das Lighthouse-Tracking zurückführen, sodass wir daran wenig ändern können, außer den Nutzer darauf hinzuweisen, dass er oder sie bitte im Trackingbereich bleiben mögen. Anderweitige Probleme entstanden beispielsweise dadurch, dass Tester nicht wieder in die

Anwendung getappt sind, nachdem sie die Aufnahme gestartet hatten. Dann funktionieren die VR-Controller immer noch, allerdings wird der Pedal Input nicht an das Programm weitergeleitet.
Ein weiteres häufig angesprochenes Feld ist die Informationsübertragung an den Nutzer. So konnten die Tester nicht sehen, wie schnell sie fuhren, bei welcher Drehzahl sie aktuell waren oder in welchem Gang sie fuhren. Die Anzeige für die technischen Daten war zu dieser Zeit schon in Arbeit, allerdings noch nicht fertig. Die Informationsübertragung wurde durch die später eingebundene Anzeige und den beschrifteten Schaltkasten bei der Kupplung deutlich verbessert.
Der letzte Kritikpunkt war der Sound des Panzers. Das Motorengeräusch war zwar passend, jedoch viel zu laut. Das Menü bot auch noch keine Einstellungsmöglichkeiten dafür. Als Konsequenz daraus haben wir speicherbare Optionen im Menü-Handbuch eingerichtet.

Zweiter Testdurchlauf:
Im zweiten Testdurchlauf waren Steffen Fietkau, Curtis Radel (22.01.), Kieran Berg, Bernhard Hansen, Laurin Holla und Hakon Engelbrecht (22.01.) unsere Tester.

Stand des Builds:
- Fahrzeugmechaniken komplett implementiert
- Technische Daten per Rotationsnadel angezeigt
- verbessertes Panzerdesign (Chassis)
- Menühandbuch mit Optionen & Steuerung implementiert
- einfache Testmap (Rampe, Unebenheiten und einige Assets) (21.01.)
  designte Map (Grabenlabyrinth, Rampe, Unebenheiten, weitere Hindernisse, visuell gestaltet) (22.01.)
- kein ausgestalteter Innenraum des Panzers

Auch der zweite Testdurchlauf hatte sehr gemischtes Feedback. Es wurde positiv angemerkt, dass die Steuerung jetzt ingame erklärt wird (mit Vorbehalten - siehe später) und das Fahrgefühl gut ist. Die Anzeige der technischen Daten hat geholfen, den Panzer in seiner Bewegung besser einzuschätzen. Dass das Menü in einem greifbaren Handbuch an der Seite des Panzers ist, wurde ebenfalls sehr positiv angemerkt.
Aber auch jetzt wurden viele Verbesserungsvorschläge und Kritiken angebracht, von denen die meisten aber schon eingeplant waren.
Ein neues Problem, welches in den Tests auftrat, war die Performance. Allerdings konnten wir das Problem extern lokalisieren. SteamVR hatte zur Testzeit einen Bug und ließ mehrere VR-Spielinstanzen parallel laufen. Ein Neustart selbiger Anwendung konnte die Probleme bezüglich Performanz beheben.
Ein weiterer Punkt, der schon mal aufgefallen ist, war das Flackern von Schatten in der Welt. Die Ursache des Flimmerns konnten wir leider nicht identifizieren, allerdings konnten wir das Problem beheben, indem wir den Schattenwurf des Richtungslichts in der Map ausgeschaltet haben. Jetzt haben viele Assets zwar keine Schatten mehr, allerdings flimmert die Welt nicht mehr. Einen Tausch, den wir gewillt waren, einzugehen.

Auch wurde die schlechte Auflösung der Schatten kritisiert. Hier haben wir uns aus zwei Gründen dafür entschieden, das Thema nicht weiter zu behandeln. Erstens wollten wir die Performance nicht zu sehr mit einer hohen Auflösung belasten und die alternative Lösung, die vorgeschlagen wurde, war, die Schatten im Panzerinnenraum auszuschalten. Dies hätte jedoch die räumliche Wahrnehmung innerhalb des Panzers deutlich verschlechtert.
Auch wurde wieder die Steuerung kritisiert. Dieses Mal weniger, dass sie nicht verständlich war, sondern vielmehr, dass man die Bedienelemente, welche erklärt wurden, nicht im Panzer identifizieren konnte. Hier gab es deutliche Unterschiede zwischen Testern mit und ohne Führerschein. Deshalb haben wir eingebaut, dass wenn die Steuerung geöffnet wird, die Nummern der Bedienelemente über diesen im Panzer erscheinen. Dies sorgt für eine eindeutige Zuordbarkeit der Hebel und Pedalen.
Außerdem wurde das nach wie vor fehlende Design der Map und des Panzer Innenraums angemerkt. Bei den Tests am 22.01., bei denen schon die ausgestaltete Map mit enthalten war, wurde diese positiv wahrgenommen und mit Freude erkundet.
Des Weiteren wurde noch vorgeschlagen, dass das Sounddesign verbessert werden könnte. Über Soundeffekte könnte das Nutzerfeedback noch verbessert werden. Ob zum Beispiel das Fahrzeug kaputt geht oder wie hoch die Drehzahl ist. Es kam unter anderem vor, dass die Tester nicht mitbekommen haben, dass der Panzer kaputt gegangen ist und sich dann gewundert haben, warum sie nicht mehr weiterfahren können. Als Konsequenz haben wir einen “Knall”-Sound eingebunden, der dem Fahrer mitteilt, dass der Motor kaputt ist.
Auch gab es den Vorschlag einen Reperatur-Button einzufügen, dass man den Panzer im Feld reparieren kann, damit man den Panzerparkour nicht von Vorne durchfahren muss. Diesen Vorschlag haben wir nicht umgesetzt, da ein Motorschaden nicht einfach im Feld repariert werden kann. Wenn man das Fahrzeug nicht richtig bedient, fährt der Panzer auch nicht lange.

Zwei Dinge sollten noch erwähnt werden, die wir erst durch die Tests richtig auf den Schirm bekommen haben. Beide hängen miteinander zusammen. Zum einen, dass die meisten Tester vor allem mit der Debug-Ansicht beziehungsweise der "Kommandanten-Sicht" gefahren sind, bei dem der Panzerchassis ausgeblendet wird. Dies hat vor allem den Grund, dass man dadurch viel mehr von der Umgebung mitbekommt. In echt gab es dafür den Kommandanten, der dem Fahrer die nötigen Informationen durch gab. Daher haben wir uns dafür entschieden, diese “Kommandanten-Sicht” im Simulator zu lassen, obwohl sie ahistorisch ist. Sie ist als Knopf ein rein optionales Feature, welches niemand verwenden muss. Ein großer Nachteil dieser Ansicht ist, dass bei fast allen Testern Motion Sickness auftrat (teilweise sogar sehr stark). In der normalen Ansicht durch den kleinen Lichtschlitz hatte hingegen fast keiner ein Problem damit.

Kompilieren:

Entwicklung:

Die Entwicklungsumgebung, in der der Simulator entwickelt wurde, ist die Unreal Engine 5.4 gewesen. Weiterhin benötigt man die SteamVR Applikation, um den Charakter in VR richtig steuern zu können.

Da wir einen Panzerfahrer-Simulator erstellt haben, werden Pedalen als Hardware beötigt. Hierfür wurde das Logitech G29 Lenkrad mit Pedalen genutzt. Um die Inputs richtig verarbeiten zu können, musste außerdem das Logitech G-Hub installiert und gestartet werden.

Als VR-Equipment haben wir die Vive 2 Pro mit den dazugehörigen Motion Controllern genutzt.

Als Plug-ins musste bei der Unreal Engine das Chaos Vehicle Plugin installiert werden und der Raw Input Plugin aktiviert werden.

Zum Modellieren der eigenen Assets wurde Blender Version 4.3 verwendet. Um dem Panzer ein passendes Skelett zu geben, wurde ein Unreal Engine 4 Rigging Addon für Blender genutzt. Damit die Blender Objekte korrekt in die Engine geladen werden, haben wir die Modelle als fbx-Datei exportiert und in die Unreal Engine geladen.

Die Texturierung der selbst modellierten Assets wurde ebenfalls in Blender vollzogen. Dafür benötigte Bildmaterialien wurden mit Gimp 2.10.38 erstellt.

Für das builden einer Anwendung in Unreal gibt es die folgenden Schritte zu beachten:

1. Im ausklappbaren Menü “Plattformen” die Zielplattform auswählen. In unserem Fall ist dies Windows.
2. Danach die binäre Konfiguration auswählen. Dies sollte zum Ende “Shipping” bzw. auf deutsch “Versand” sein
3. Danach auf “Package projekt” bzw. “Projekt verpacken” klicken
4. Einen passenden Speicherort aussuchen

Wenn diese Schritte befolgt sind, wird das Projekt als build an dem gewählt Ort abgespeichert.

Um das gebuildete Projekte zu starten muss dann nur noch die dort gespeicherte .exe ausgeführt werden.

Starten als Nutzer:

Die per Link downloadbare ZIP-Datei muss in einem beliebigen Verzeichnis entpackt werden.

Bevor die exe-Datei gestartet werden kann, muss zuvor Steam VR installiert und gestartet werden. Ebenfalls wird Logitech G-Hub benötigt, um den Pedalinput korrekt an das Programm weiterzuleiten. Dieses muss ebenfalls installiert und gestartet werden. **Danach** kann die exe Datei ausgeführt werden.

(Hinweis: Sollte der Pedal Input nicht korrekt erkannt werden, liegt es in der Regel daran, dass die Simulationsanwendung nicht im Vordergrund ausgeführt wird. Ein erneutes Reintappen in die Anwendung sollte dieses Problem beheben. Ob der Pedal Input korrekt erkannt wird, kann daran erkannt werden, ob sich die Pedalen im Panzer bewegen, wenn die Pedalen gedrückt werden.)

Fazit:

Nico Müller:
Insgesamt fand ich, dass das VR Praktikum eines der besten Module im gesamten CGT-Studiengang ist. Es ermöglicht selbständige Projektarbeit, bei dem man ein Thema bearbeiten kann, welches einen selbst interessiert. Unsere kleine Gruppe hat gut funktioniert. Ich konnte mich auf René verlassen. Ich konnte davon ausgehen, dass wir beide unsere Absprachen einhalten. Wir beide haben gute Ideen mit eingebracht und lösungsorientiert zusammengearbeitet. Im Rahmen des Projektes konnten wir uns mit neuen uns unbekannten Aspekten der Unreal Engine 5 auseinandersetzen, was für ein tieferes Verständnis für die Engine gesorgt hat.

Wir konnten alle verbindlichen Ziele aus dem Exposé erfolgreich umsetzen. Auch einige der optionalen Features haben Einzug erhalten und haben das Endprodukt bereichert. Die Fahrzeugsteuerung konnte wie gewünscht implementiert werden. Zwar gab und gibt es nach wie vor ein paar Probleme mit der genutzten Fahrzeugschnittstelle, allerdings sind diese eher klein und lassen sich nur durch eine vollständige Überarbeitung/Neuaufsetzung der Schnittstelle und der dahinter stehenden Physik beheben.
Die Gestaltung am Ende der Bearbeitungszeit hat nochmal die Atmosphäre des Simulators erheblich verbessert.
Während der Entwicklungsphase sind mir noch viele Ideen gekommen, wie man das Spielerlebnis noch verbessern und immersiver machen könnte. Das sind jedoch alles kleine Features gewesen, welche aus Zeit gründen einfach nicht umgesetzt werden konnten. Darunter fallen Dinge wie verfeinertes Feedback (z.B. Sicherung bei der Handbremse wird gedrückt, wenn die Handbremse bedient wird) oder verbessertes Sounddesign (z.B. Motorengeräusch passt sich an die Drehzahl an).
Insgesamt bin ich mit dem Ergebnis zufrieden und kann sagen, dass das Projekt eine lehrreiche Erfahrung im Umgang mit der Entwicklung von VR Anwendungen war.

Die Playtests durch Externe konnten einige unbekannte Bugs aufzeigen. Ansonsten haben sie vor allem unseren aufgestellten Entwicklungsplan bestätigt. Im Nachhinein betrachtet hätte man für die Tests einige Features wie die Ingame-Steuererklärung und das Mapdesign früher umsetzen sollen, da es zu besseren und ausgiebigen Tests geführt hätte, allerdings lagen unsere Prioritäten während der Entwicklungsphase woanders. Hinter dieser Prioritätensetzung stehe ich auch nach wie vor. Um solche Features früher einzubinden, hätte es mehr Arbeitskraft benötigt. Da wir aber ein kleine Gruppe, bestehend aus nur zwei Leuten, sind, ließ sich diese Priorisierung nur schwer vermeiden.

René Graap:

Das VR-Praktikum war eine, meiner Meinung nach, sehr interessante Übung. Ich finde, dass unsere kleine Gruppengröße für unser Projekt sehr gut geeignet war. Dadurch, dass Nico und ich nur zu zweit waren, konnten wir fast alles zu zweit bearbeiten und auch mit der Absprache zwischen uns gab es keine Probleme. Wir konnten beide gute Einfälle in das Projekt einbringen

und haben uns an unbekannten Problemen zusammen den Kopf zerbrochen. Auch die Hilfe von Marjan beim richtigen Zuweisen der Pedalen hat sehr geholfen.

Wir konnten alle nicht optionalen Ziele, die wir uns beim Expose gesetzt haben, erfüllen und hatten sogar noch genug Zeit, einige der optionalen Ziele umzusetzen. Da wir aber nur eine Zweier Gruppe waren, hat es leider nicht für alle optionalen Ziele und Ideen gereicht, die uns noch während der Entwicklung kamen. Dennoch würde ich sagen, dass wir uns tief mit der Unreal Engine in VR auseinandergesetzt haben und einiges gelernt haben. Vor allem das Unreal Engine ChaosVehicle Plug-in hat uns häufiger Probleme bereitet, deren Behebung allerdings vollkommen den Rahmen des Praktikums gesprengt hätte. So mussten wir uns dann mit Workarounds auseinandersetzen.
Dass die Gestaltung der Map erst am Ende der Entwicklung erfolgt ist und nicht schon vorher, ist schade, aber wir hatten unsere Prioritäten vorher darauf gelegt, einen voll funktionsfähigen Panzer zu erstellen. Aber am Ende hat die Ausgestaltung der Map und des Panzers den Simulator um einiges besser gemacht, wie wir es auch als Feedback von den letzten Playtestern bekommen haben.
Auch die Playtests waren sehr sinnvoll, da wir dadurch einige uns vorher unbekannte Bugs finden konnten. Weiterhin hatten uns einige Playtests auch auf nötige Verbesserungen in unserer Readme aufmerksam gemacht. Ein Großteil der Rückmeldungen waren glücklicherweise Dinge, die wir uns sowieso schon vorgenommen hatten (z.B. bessere Erklärung der Steuerung). Dass dies auch genannt wurde, hatte uns also nur mehr in unserem Entwicklungsplan bestätigt.

Insgesamt kann ich also sagen, dass das Praktikum eine gelungene Übung war, die zumindest mir, trotz der Unreal Probleme, sehr viel Spaß bereitet hat.

Verwendete Assets:

Stylized Nature Pack: https://www.fab.com/listings/b066de06-73b8-4fbe-b30c-468f5bcf7575
Sitz: https://www.fab.com/listings/f8d3a3b6-e8cf-4999-b24f-0443f14bfe36
Rucksack: https://www.fab.com/listings/9973d782-cad6-451c-bc4c-b407f209b9c3
Gewehr: https://www.fab.com/listings/8f0b2497-cf5c-47d7-b829-5c337fd0d763
Holzkiste: https://www.fab.com/listings/5798fb14-e141-4d89-9401-ac101b0b2f0b
Waffenkiste: https://www.fab.com/listings/3497e536-7f0c-4348-8932-985780cca82c
Hänger für Rucksack: https://www.fab.com/listings/e3730d66-f525-4c9d-8c5d-eaaf75ddb675
Kupplungshebel: https://drive.google.com/file/d/1x0yfsfj_pkZ_muMEZfktDkjRKT-G948k/view
Sounddatei Panzermotor: https://pixabay.com/sound-effects/tank-moving-143104/

Blender Rigging Plug-in für Unreal:
https://continuebreak.com/creations/ue4-vehicle-rigging-addon-blender/